TRAITEMENT DE DONNÉES · FICHE DE RÉVISION · COURS 2

Sac de mots, TF-IDF et K-Means

Notebooks « Bag of words » et « K-Means » : représenter des textes, puis les regrouper

À RETENIR EN PRIORITÉ

- **Non supervisé** : on ne connaît ni les classes ni leur nombre ; on regroupe par ressemblance (*clustering*).
- **Sac-de-mots** = histogramme des occurrences des mots du dictionnaire (vecteur de la taille du vocabulaire, **creux**). Ni ordre, ni syntaxe.
- Vocabulaire **appris sur le train** (`fit`), test seulement `transform` : un mot hors vocabulaire est **ignoré**.
- **TF** = n(t,doc) / Card(doc) · **IDF** = log(M / nb de docs contenant t) : 0 si t est partout, log M s'il est dans un seul doc.
- Plus gros gain : la **suppression des stop words** (precision@1 0,49 → 0,75). **BM25** > TF-IDF ici (longueurs très variables).
- **K-Means** minimise $J = \sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$: affecter au centre le plus proche, puis centre = **barycentre**, jusqu'à stabilité.
- Minimum **local** → plusieurs initialisations (`n_init`, k-means++). J **décroît toujours** avec K → choisir K par le **coude** ou la **silhouette**.
- K-Means suppose des variables **à la même échelle** (`StandardScaler`) et des groupes **convexes** (échec sur cercles concentriques).

1. Du texte au vecteur : one-hot et sac-de-mots

Donnée catégorielle à K valeurs → vecteur **one-hot** « 1 parmi K » plutôt qu'un entier 1…K :

- toutes les catégories sont **à la même distance** (pas d'ordre artificiel) ;
- la **moyenne** de ces vecteurs donne la **proportion** de chaque catégorie.

Sac-de-mots (extension du one-hot) : composante *i* = nombre d'occurrences du *i*-ème mot du dictionnaire dans le document.

Jeu de données : articles du *Monde* (MLSUM), **6 rubriques × 2 000** articles → train 9 600, test 2 400 (1 600 par catégorie dans le train).

Vecteurs creux : très grands (≈ 107 000 mots bruts), presque tous nuls → problème de **mémoire** (géré par `scipy.sparse`) et de **dimension** (il faudrait beaucoup de données, sinon **sur-apprentissage**).

`CountVectorizer` passe déjà tout en **minuscules** par défaut.

MÉTHODE : VECTEUR DE COMPTAGE D'UNE PHRASE

1. Tokeniser, minuscules, retirer les stop words.
2. Pour chaque mot **du vocabulaire appris sur le train** (ordre des colonnes), compter ses occurrences.
3. Les mots absents du vocabulaire sont **ignorés** (aucune colonne ajoutée).

Exemple : vocabulaire (chat, chien, mange, souris) · « Le chat mange la souris, puis le chat dort. » → chat 2, chien 0, mange 1, souris 1 : **(2, 0, 1, 1)** — « puis » et « dort » sont hors vocabulaire.

2. fit sur le train, transform sur le test

```
vec = CountVectorizer()
vec.fit(train["text"])          # apprend le vocabulaire
X = vec.transform(train["text"])
X.shape   # (9600, 107457) = (nb docs, taille vocab)
X_test = vec.transform(test["text"])  # vocab figé
vec.vocabulary_                 # dict mot -> colonne
vec.get_feature_names_out()     # mots, dans l'ordre
```

Mots hors vocabulaire (*OOV*) : ≈ **20 %** du vocabulaire du test n'existe pas dans celui du train — surtout des **noms propres**.

Parade : **méthodologique, en amont** → un train **le plus large et représentatif possible**. Jamais apprendre le vocabulaire sur le test.

3. Évaluer une représentation (sans classifieur supervisé)

precision@k : pour chaque article du test, ses **k plus proches voisins dans le train** (similarité **cosinus**) ; part de voisins de même catégorie, **moyennée sur le test**. Baseline aléatoire : **1/6 ≈ 0,167**.

t-SNE : projection 2D qui préserve les voisinages, après **SVD tronquée**. Outil de **visualisation**, pas de clustering : il ne crée pas de groupes.

Sous-ensemble **Difficile** = International, Société, Politique : fort **recouvrement lexical**. Facile = Sport, Culture, Économie.

precision@k : voisins Sport, Sport, Culture, Sport, Politique d'un article Sport → p@1 = **1**, p@5 = 3/5 = **0,6**.

COSINUS VS EUCLIDIENNE

cos(u, v) = u·v / (‖u‖ ‖v‖)

a = (2,0,1,1), b = (1,0,0,1), c = 2a = (4,0,2,2)
a·b = 3, ‖a‖ = √6, ‖b‖ = √2 → cos(a,b) = 3/√12 ≈ **0,87**
d(a,c) = ‖(2,0,1,1)‖ = √6 ≈ **2,45** mais cos(a,c) = **1**

Le cosinus **ignore la longueur** du document : même contenu doublé = similarité 1.

4. Améliorations successives de la représentation

Étape	Comment	Vocabulaire	p@1	p@5	p@10
1. Brut	`CountVectorizer()`	107 457	0,487	0,432	0,399
2. + stop words	`stop_words=stwf` (157 mots NLTK)	107 330	**0,753**	0,710	0,683
3. + filtre numérique	tokenizer maison, `not t.isdigit()`	105 749	0,757	0,709	0,683
4. + stemming	`SnowballStemmer("french")`, `min_df=5`	18 871	0,765	0,725	0,703
5. + TF-IDF	`TfidfVectorizer(..., min_df=5)`	18 871	0,749	0,722	0,708
6. BM25 (à la place)	`CountVectorizer` + `BM25Transformer`	18 871	**0,784**	**0,744**	**0,732**

Stop words : articles, auxiliaires et leurs conjugaisons, mots de liaison (le, la, être, avoir, mais, sur…) : rôle grammatical, aucune information sur le sujet.

Filtre numérique : dates, scores… n'ont pas de sens thématique. Le motif `\b\w\w+\b` ignore les tokens d'1 caractère ; `isdigit()` ne retire que les tokens **purement** numériques :
« Les 2 buts du match à 19h, en 2019 » → `['buts', 'match', '19h']`.

Brancher sa fonction : `CountVectorizer(tokenizer=f, token_pattern=None)`.

	Lemmatisation	Racinisation (stemming)
Garde	la forme canonique (infinitif, singulier)	la racine, sans préfixes/suffixes
Mot réel ?	Oui	En général **non** (« étudiants » → « étudi »)
Exige	connaissance lexicale (verbe ou nom ?)	des règles (Porter, Snowball)
Cours	non utilisée	**utilisée** : `SnowballStemmer`

PIÈGES

`min_df=5` retire les termes présents dans **moins de 5 documents** du train (pas un pourcentage, pas une longueur de document). Conséquence : tous les termes présents dans exactement 5 documents sont **ex-æquo à l'IDF maximal** (8,38). Les IDF les plus faibles sont des mots génériques (« plus », « fait », « tout », « deux ») qui ont échappé aux stop words.

5. Pondération TF-IDF

TF : f(t, doc) = n(t, doc) / Card(doc) | **IDF(t) = log(M / Card{doc : t ∈ doc})** | **TF-IDF = f(t, doc) × IDF(t)**

- **TF** : normalise par la longueur → même comptage, document plus long = poids plus faible.
- **IDF** : compte les **documents** (pas les occurrences). Terme partout → IDF = log 1 = **0** ; dans un seul doc → **log M**. IDF élevé = terme **rare, discriminant**.

MÉTHODE : CALCULER UN TF-IDF À LA MAIN (LOG NÉPÉRIEN)

Corpus M = 4 : d1 = « but match but équipe », d2 = « match équipe joueur », d3 = « vote match loi », d4 = « vote loi match député ».

```
terme     docs   IDF = ln(4/docs)    TF-IDF
but        1     ln 4   = 1,386      d1 : 2/4 x 1,386 = 0,693
match      4     ln 1   = 0          0 partout
équipe     2     ln 2   = 0,693      d1 : 1/4 x 0,693 = 0,173   d2 : 1/3 x 0,693 = 0,231
vote       2     ln 2   = 0,693      d3 : 1/3 x 0,693 = 0,231   d4 : 1/4 x 0,693 = 0,173
```

Erreurs classiques : oublier de diviser par la longueur (but : 2,77), utiliser $\log_{10}$ (0,30), compter les occurrences dans l'IDF.

À SAVOIR

`TfidfVectorizer` de scikit-learn utilise en réalité une variante (IDF lissé ln((1+M)/(1+df)) + 1, comptages bruts puis normalisation L2 du vecteur) : c'est elle qui donne l'IDF max 8,38 = ln(9601/6) + 1. Pour un calcul à la main, prendre la **formule du cours**. Attribut des IDF appris : `vectorizer.idf_`.

6. BM25 : une pondération alternative

$$\text{BM25}(t,d) = \text{IDF}_{\text{BM25}}(t) \cdot \text{tf}\cdot(k_1+1) \,/\, (\,\text{tf} + k_1\cdot(1 - b + b\cdot|d| \,/\, \text{avgdl})\,)$$

$$\text{IDF}_{\text{BM25}} = \log(1 + (N - \text{df} + 0{,}5)/(\text{df} + 0{,}5))$$

- **Saturation** de la fréquence : passer de 1 à 2 occurrences compte plus que de 10 à 11 (TF-IDF est linéaire).
- **Normalisation explicite par la longueur** |d| / avgdl (longueur moyenne du train).
- Hyperparamètres : k_1 (saturation, 1,2–2,0) et **b** (poids de la longueur, ≈ 0,75). Référence d'Elasticsearch / Lucene.
- Pas de `BM25Vectorizer` dans scikit-learn : le cours écrit un `BM25Transformer` branché après un `CountVectorizer` dans un `Pipeline`, **même prétraitement** que TF-IDF (alternative, pas cumul).
- Gagne ici car les articles ont des **longueurs très variables** (300 à 45 000 caractères) ; sur des titres courts, aucun avantage → toujours **mesurer**.

7. L'algorithme des K-Moyennes (K-Means)

Points de référence : on affecte chaque point à la référence **la plus proche**. Frontière entre deux références = droite **équidistante** (médiatrice) ; pour toutes les paires → **diagramme de Voronoï**.

Critère (somme des inerties) :

$$J = \sum_{k=1..K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Inertie d'un groupe faible = groupe compact.

Paramètres appris : les **centres** μ_k et l'**assignation** des points. **Hyperparamètres** : K, `n_init`.

ALGORITHME

1. **Initialisation** : K observations tirées au hasard **parmi les données** (scikit-learn : **k-means++**, centres initiaux éloignés).
2. **Assignation** : chaque point → centre le plus proche (Voronoï des centres).
3. **Mise à jour** : μ_k = **moyenne** des points du groupe.
4. Recommencer 2–3 jusqu'à ce que les **centres** ou **J** ne bougent plus significativement (seuil relatif).

Pourquoi la moyenne ? Groupes fixés : $\nabla_{\mu_k} J_k = -2 \sum_{i \in C_k}(x_i - \mu_k) = 0 \Rightarrow \mu_k = (1/n_k) \sum x_i$.

MINIMUM LOCAL

J décroît à chaque itération, mais vers un minimum qui peut être **local** selon les centres initiaux. On relance **plusieurs fois** (`n_init`, dizaines à centaines) et on garde le **J final minimal**. `n_init` ≠ nombre d'itérations (`max_iter`).

MÉTHODE : UNE ITÉRATION DE K-MEANS À LA MAIN

P1(0,0), P2(1,0), P3(0,2), P4(4,4), P5(5,4), P6(4,6) · K = 2 · centres initiaux μ1 = P1, μ2 = P3.

```
       d(.,mu1=(0,0)) d(.,mu2=(0,2)) groupe
P1          0,00           2,00         1
P2          1,00           2,24         1
P3          2,00           0,00         2
P4          5,66           4,47         2
P5          6,40           5,39         2
P6          7,21           5,66         2
Mise à jour : mu1 = (0,5 ; 0)   mu2 = ((0+4+5+4)/4 ; (2+4+4+6)/4) = (3,25 ; 4)
Itération 2 : P3 -> d(mu1) = 2,06 < d(mu2) = 3,82 : P3 passe dans le groupe 1
              mu1 = (1/3 ; 2/3)   mu2 = (13/3 ; 14/3)   puis plus rien ne bouge
Inertie finale : groupe 1 = 5/9 + 8/9 + 17/9 = 30/9 ; groupe 2 = 30/9 ; J = 60/9 = 6,67
```

Pièges : oublier le **carré** (somme des distances = 6,13), oublier un point dans la moyenne.

8. Choisir K et évaluer un clustering

J(K) décroît toujours avec K (J = 0 si K = nombre de points) : on ne minimise pas J pour choisir K.

Méthode du coude :

$\Delta_{rel}(K) = (J(K-1) - J(K)) / J(K-1)$

Le coude = dernier K qui apporte une forte baisse ; au-delà, gain marginal.

J(1..5) = 1000, 600, 150, 130, 120 → Δ_{rel} = 0,40 ; **0,75** ; 0,13 ; 0,08 → coude à **K = 3**.

Validation : ajuster les centres sur le train, comparer l'inertie **moyenne par point** sur train et validation (`-clust.score(X)/len(X)`) : courbes superposées = vraie structure, pas un hasard d'échantillonnage.

Silhouette (sans vérité terrain) :

s(i) = (b(i) – a(i)) / max(a(i), b(i))

- a(i) : distance **moyenne** aux autres points de **son** cluster ;
- b(i) : distance **moyenne aux points** du cluster voisin le plus proche (pas à son centre) ;
- ≈ 1 bien placé, ≈ 0 frontière, < 0 mal affecté. Score global = moyenne. Non défini pour K = 1.
- K trop grand (cluster coupé en deux) → silhouette **baisse**.

a = 0,8, b = 2,0 → s = 1,2/2 = **0,6** · a = 1,5, b = 1,0 → s = −0,5/1,5 = **−0,33**

ARI (avec vérité terrain) : 1 = partitions identiques à une renumérotation près, 0 = hasard. Seulement sur données synthétiques/étiquetées ; sur données réelles, la **silhouette** est la référence.

9. Limites de K-Means

Hypothèse implicite	Quand elle casse	Remède
Variables **comparables** (distance euclidienne)	Une variable ×30 sans information domine la distance : K-Means coupe selon elle (ARI ≈ 0)	`StandardScaler` (moyenne 0, écart-type 1) → ARI ≈ 1
Groupes à peu près **convexes**	Cercles concentriques (`make_circles`) : ARI ≈ 0, même avec un grand `n_init`	Variable $x^2 + y^2$ (idée du kernel trick) ou **clustering spectral** (graphe de plus proches voisins)

Autres familles évoquées : DBSCAN (densité), mélanges de gaussiennes, SVM à noyau (classification).

10. Mémo scikit-learn

- `CountVectorizer` / `TfidfVectorizer` : `fit` apprend le vocabulaire, `transform` → matrice creuse (docs × vocab) ; `.vocabulary_`, `.idf_`. `cosine_similarity(A, B)`, `euclidean_distances(A)`.
- `KMeans(n_clusters=K, n_init=10).fit(X)` → `.labels_`, `.cluster_centers_`, `.inertia_` (J) ; `.fit_predict(X)` → labels ; `.score(X)` = **−inertie**. `silhouette_score(X, labels)`, `adjusted_rand_score(y_vrai, labels)`.